

# UNIVERSIDAD NACIONAL DE ENTRE RÍOS

Facultad de Ingeniería

Carrera: Licenciatura de Ciencia de Datos

Materia: Fundamentos de Algoritmos y Estructuras de Datos

Título del Trabajo: **Aplicaciones de estructuras jerárquicas y grafos**

Equipo del Docente:

Dr. Javier E. Diaz Zamboni

Bioing. Jordán F. Insfrán

Bioing. Diana Vertiz del Valle (de licencia)

Esp. Bioing. Juan Francisco Rizzato

Dr. Bioing. Feliciano Franco

Integrantes del equipo:

Franco Acevedo

Franco Tomiozzo

Comisión: 1

Fecha de entrega: 08/06/2026

Link del repositorio:

<https://github.com/RENCO12BOT/AyED2026c1-Acevedo-Tomiozzo-.git>



Universidad Nacional  
de Entre Ríos

## 1. INTRODUCCIÓN

El simulador original de la sala de emergencias atendía pacientes en orden de llegada (**FIFO simple**). El objetivo de este trabajo fue reemplazar esa lógica por una estructura que garantice que el paciente más crítico siempre sea el primero en ser atendido, independientemente del orden de llegada.

La solución implementada es una **Cola de Prioridad** basada en un **Montículo Binario Mínimo (Min-Heap)**. Esta estructura mantiene el elemento de menor prioridad numérica (mayor urgencia clínica) siempre en la raíz, permitiendo extracción en  **$O(\log n)$** . Para empates de prioridad, se aplica criterio FIFO mediante un contador de llegada, lo que garantiza equidad y determinismo en el ordenamiento.

La implementación quedó dividida en tres capas bien separadas: **monticulo\_binario.py** (estructura genérica base), **Cola\_Prioridad.py** (encapsula el montículo y agrega el criterio de desempate) y **test\_main.py** (aplicación concreta: el simulador de sala de emergencias).

## 2. DESARROLLO DE LOS PROBLEMAS

### A. Análisis y planteo

Para que el paciente más crítico siempre sea el primero en salir, se necesita una estructura que mantenga el mínimo accesible en  **$O(1)$**  y que permita inserciones y extracciones eficientes. Se compararon las principales alternativas:

Comparando las principales alternativas:

| Estructura        | Inserción         | Extraer mínimo | ¿Adecuada?                 |
|-------------------|-------------------|----------------|----------------------------|
| Lista desordenada | $O(1)$            | $O(n)$         | No, Búsqueda lineal        |
| Lista ordenada    | $O(n)$            | $O(1)$         | No, Inserción lenta        |
| Min-Heap          | $O(\log n)$       | $O(\log n)$    | Sí, Es óptimo              |
| Árbol BST         | $O(\log n)$ prom. | $O(\log n)$    | Posible, pero más complejo |

Se eligió el **Min-Heap** porque es la estructura estándar para colas de prioridad dinámicas. En un escenario donde los pacientes llegan y son atendidos continuamente, el  **$O(\log n)$**  tanto en inserción como en extracción es ideal.



## B. Implementación / Resolución

La implementación quedó dividida en tres capas bien separadas:

### MonticuloBinario

Estructura genérica base. Usa una lista interna donde el índice **0** es un valor de relleno. Para cualquier nodo en posición **i**: su padre está en **i//2**, hijo izquierdo en **2i** y derecho en **2i+1**.

```
def infiltArriba(self, i):
    # sube el elemento hasta que este en su lugar correcto
    while i // 2 > 0:
        if self.listaMonticulo[i] < self.listaMonticulo[i // 2]:
            self.listaMonticulo[i], self.listaMonticulo[i // 2] = \
                self.listaMonticulo[i // 2], self.listaMonticulo[i]
            i //= 2
        else:
            break
```

```
def eliminarMin(self):
    if self.tamanoActual == 0:
        return None
    valorSacado = self.listaMonticulo[1]
    # ponemos el ultimo elemento en la raiz y despues lo bajamos
    self.listaMonticulo[1] = self.listaMonticulo[self.tamanoActual]
    self.tamanoActual -= 1
    self.listaMonticulo.pop()
    if self.tamanoActual > 0:
        self.infiltAbajo(1)
    return valorSacado
```

### ColaPrioridad: desempate FIFO

Cada elemento se inserta como la tupla (prioridad, contador, dato). Como las tuplas en Python se comparan elemento a elemento, si dos pacientes tienen el mismo riesgo, el que llegó primero (menor contador) tiene prioridad:

```
def insertar(self, prioridad, dato):
    # guardamos la prioridad y el orden de llegada juntos para comparar
    clave = (prioridad, self._contador)
    self._monticulo.insertar((clave, dato))
    self._contador += 1
```



## Integración al simulador

Las modificaciones al simulador (**test\_main.py**) fueron mínimas y se limitaron a reemplazar dos líneas:

- `cola_de_espera.append(paciente) → cola_de_espera.insertar(paciente.get_riesgo(), paciente)`
- `cola_de_espera.pop(0) → cola_de_espera.extraer()`

El resto del código original se mantuvo sin cambios. Esta separación entre estructura y aplicación permitió además reutilizar la misma ColaPrioridad en el Problema 3 (algoritmo de Prim) sin ninguna modificación.

## Precondiciones y Postcondiciones

Se definen pre y postcondiciones para los métodos principales de ambos módulos:

| Método                                              | Precondición                                                        | Postcondición                                                                       |
|-----------------------------------------------------|---------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| <b>insertar(k)</b><br>(MonticuloBinario)            | <b>k</b> no es <b>None</b> y es comparable con los demás elementos. | <b>k</b> queda en el <b>heap</b> ; se mantiene la propiedad de <b>min-heap</b> .    |
| <b>eliminarMin()</b>                                | <b>tamanoActual &gt; 0</b><br>( <b>heap no vacío</b> )              | Se devuelve la <b>raíz (mínimo)</b> ; el heap sigue siendo válido.                  |
| <b>infilArriba(i)</b>                               | $1 \leq i \leq \text{tamanoActual}$                                 | El elemento en <b>i</b> cumple la propiedad de <b>heap</b> respecto a su padre.     |
| <b>infilAbajo(i)</b>                                | $1 \leq i \leq \text{tamanoActual}$                                 | El elemento en <b>i</b> cumple la propiedad de <b>heap</b> respecto a sus hijos.    |
| <b>insertar(prioridad, dato)</b><br>(ColaPrioridad) | La prioridad es entero $\geq 1$ ; dato no es <b>None</b> .          | El dato queda insertado con su prioridad y contador <b>FIFO</b> .                   |
| <b>extraer() (ColaPrioridad)</b>                    | Ninguna; si la cola está vacía, retorna <b>None</b> .               | Retorna el dato con mayor prioridad (menor número). En empate, el que llegó antes.. |
| <b>ver_proximo()</b><br>(ColaPrioridad)             | Ninguna.                                                            | Retorna el próximo dato sin modificar la cola, o <b>None</b> si está vacía.         |



Las validaciones implementadas lanzan las siguientes excepciones:

- **ValueError** — insertar None, prioridad negativa, lista con None en construirMonticulo.
- **TypeError** — prioridad no numérica, parámetro no lista en construirMonticulo.

## C. Resultados y pruebas

Se realizaron pruebas de uso sobre cada módulo individualmente para corroborar su funcionamiento antes de integrarlos al simulador. No se usaron frameworks de testing: cada prueba imprime el resultado obtenido junto al resultado esperado.

### TEST: Sala de Emergencia

Se corrió el simulador con **20 ciclos**. La captura muestra un momento clave: un paciente de riesgo **1 (crítico)** que llega cuando ya hay pacientes de riesgo **2** o **3** en la cola, y es atendido primero en el siguiente ciclo de atención. Esto confirma que la **ColaPrioridad** funciona correctamente integrada al simulador.

```
05/06/2026 13:35:05
*****
Se atiende el paciente: Antonio Mendez   -> 1-crítico
*****

Pacientes que faltan atenderse: 10
    Agustina Mendez           -> 2-moderado
    Jorge Perez               -> 3-bajo
    Mariela Rodriguez         -> 3-bajo
    Agustina Rodriguez        -> 3-bajo
    Gastón Mendez             -> 3-bajo
    Jorge García              -> 3-bajo
    Andrea García             -> 3-bajo
    Mariela Colman            -> 3-bajo
    Gastón Perez              -> 3-bajo
    Andrea Colman              -> 3-bajo

_*_*_*_*_*_*_*_*_*_*_*_*_*_*_*_*_*_*_*_*_*_*_
_*_*_*_*_*_*_*_*_*_*_*_*_*_*_*_*_*_*_*_*_*_*_

05/06/2026 13:35:06
*****
Se atiende el paciente: Agustina Mendez   -> 2-moderado
*****

Pacientes que faltan atenderse: 10
    Jorge Perez               -> 3-bajo
    Mariela Rodriguez         -> 3-bajo
    Agustina Rodriguez        -> 3-bajo
    Gastón Mendez             -> 3-bajo
    Jorge García              -> 3-bajo
    Andrea García             -> 3-bajo
    Mariela Colman            -> 3-bajo
    Gastón Perez              -> 3-bajo
    Andrea Colman              -> 3-bajo
    Agustina García           -> 3-bajo
```



### 3. ANÁLISIS DE RESULTADOS ESPECÍFICOS / DISCUSIÓN

La tabla resume la complejidad temporal de cada operación. El análisis se basa en que el heap es un árbol binario completo de altura  $\lfloor \log n \rfloor$ , por lo tanto todas las operaciones que recorren la altura son  $O(\log n)$ :

| Operación                                           | Complejidad   | Justificación                                                        |
|-----------------------------------------------------|---------------|----------------------------------------------------------------------|
| <b>insertar(k)</b><br>(MonticuloBinario)            | $O(\log n)$   | <b>infilArriba</b> recorre como máximo la altura del árbol           |
| <b>eliminarMin()</b><br>(MonticuloBinario)          | $O(\log n)$   | <b>infilAbajo</b> recorre como máximo la altura del árbol            |
| <b>construirMonticulo(lista)</b>                    | $O(n)$        | Se aplica <b>infilAbajo</b> desde $n/2$ hasta 1; análisis amortizado |
| <b>insertar(prioridad, dato)</b><br>(ColaPrioridad) | $O(\log n)$   | Delega en insertar del montículo                                     |
| <b>extraer()</b> (ColaPrioridad)                    | $O(\log n)$   | Delega en <b>eliminarMin</b> del montículo                           |
| <b>ver_proximo()</b><br>(ColaPrioridad)             | $O(1)$        | Acceso directo a <b>listaMonticulo[1]</b> , la raíz                  |
| <b>ver_todos()</b> (ColaPrioridad)                  | $O(n \log n)$ | Usa <b>sorted()</b> sobre los $n$ elementos internos                 |
| <b>esta_vacia()</b> / <b>tamano()</b>               | $O(1)$        | Consulta directa del atributo <b>tamanoActual</b>                    |

Comparado con una lista simple donde encontrar el mínimo requeriría recorrer todos los elementos ( $O(n)$ ) el **heap** es claramente superior para este problema donde las operaciones se ejecutan de forma continua.

Una decisión de diseño importante fue separar el MonticuloBinario de la ColaPrioridad. Esto permite reutilizar la misma ColaPrioridad en el Problema 3 (algoritmo de Prim) sin ningún cambio, lo que demuestra el valor de diseñar estructuras genéricas.

**ver\_todos()** tiene complejidad  $O(n \log n)$  porque usa **sorted()**. Para un sistema real de alta carga este método debería eliminarse o reemplazarse, ya que solo se usa para visualización y no aporta funcionalidad crítica.

La decisión de usar **FIFO** como criterio de desempate es éticamente correcta en un contexto médico: dos pacientes con idéntico nivel de riesgo no deberían ser reordenados arbitrariamente. El contador de llegada garantiza esto de forma determinística sin costo adicional de complejidad.



También se corrigió un bug en **infiltrArriba**: en la versión original el índice avanzaba siempre aunque no hubiera swap, lo que podía hacer ascender un elemento más de lo necesario. Se corrigió agregando un **break** cuando no hay intercambio.

#### 4. DECLARACIÓN DE USO ÉTICO DE INTELIGENCIA ARTIFICIAL

##### Herramientas utilizadas:

- Claude (Anthropic), GitHub Copilot.

##### Descripción del uso:

- **Consulta teórica:** se consultó para repasar el funcionamiento del **heap** y comparar con otras estructuras para justificar la elección.
- **Asistencia en código:** se usó para identificar el bug en **infiltrArriba** y revisar la lógica del criterio de desempate con tuplas.
- **Redacción:** apoyo para mejorar la redacción del informe.

##### Declaración de responsabilidad:

Los integrantes del grupo declaramos que revisamos y validamos personalmente todo el código e información generada con **IA**. Entendemos los conceptos del informe y asumimos la responsabilidad académica por la integridad del contenido. El trabajo es producto de nuestro propio análisis.

#### 5. TRABAJO EN EQUIPO

##### Estimación inicial:

Calculamos unas **6** horas en total: **1** hora para investigar **heaps**, **3** horas de desarrollo, **1** hora de pruebas y **1** hora para el informe.

##### Distribución de tareas:

- **Franco Tomiozzo:** implementación de **MonticuloBinario** y **ColaPrioridad**, pruebas de los módulos individuales.
- **Franco Acevedo:** integración al simulador (**test\_main.py**), análisis de complejidad y redacción del informe.

##### Coordinación:



Nos dividimos los módulos por **WhatsApp** y **Discord** e integramos mediante commits en el repositorio **GitHub**. Las diferencias de criterio sobre el desempate (**FIFO vs. aleatorio**) las resolvimos optando por **FIFO**, que es lo más correcto desde el punto de vista médico.

#### **Tiempo real vs. estimado:**

Tardamos aproximadamente **7 horas**. La hora extra se fue en depurar el bug de **infilArriba** y en entender por qué el contador de desempate era necesario en los casos de empate de prioridad.

### **6. CONCLUSIONES**

Se cumplió el objetivo: el simulador ahora atiende siempre al paciente más crítico primero, con una estructura genérica reutilizable y eficiente.

Lo más valioso del trabajo fue entender en detalle cómo funciona la propiedad de heap y por qué **infilArriba** e **infilAbajo** la restauran en  **$O(\log n)$** . También aprendimos que separar la implementación de la aplicación tiene ventajas concretas: la misma **ColaPrioridad** nos sirvió en el **Problema 3** sin modificaciones.

### **7. BIBLIOGRAFÍA**

- Miller, B. & Ranum, D. Problem Solving with Algorithms and Data Structures using Python. Runestone Academy. <https://runestone.academy/>
- Cormen, T. et al. Introduction to Algorithms, 3ra ed. MIT Press, 2009.
- Repositorio del proyecto:

<https://github.com/RENCO12BOT/AyED2026c1-Acevedo-Tomiozzo-.git>

